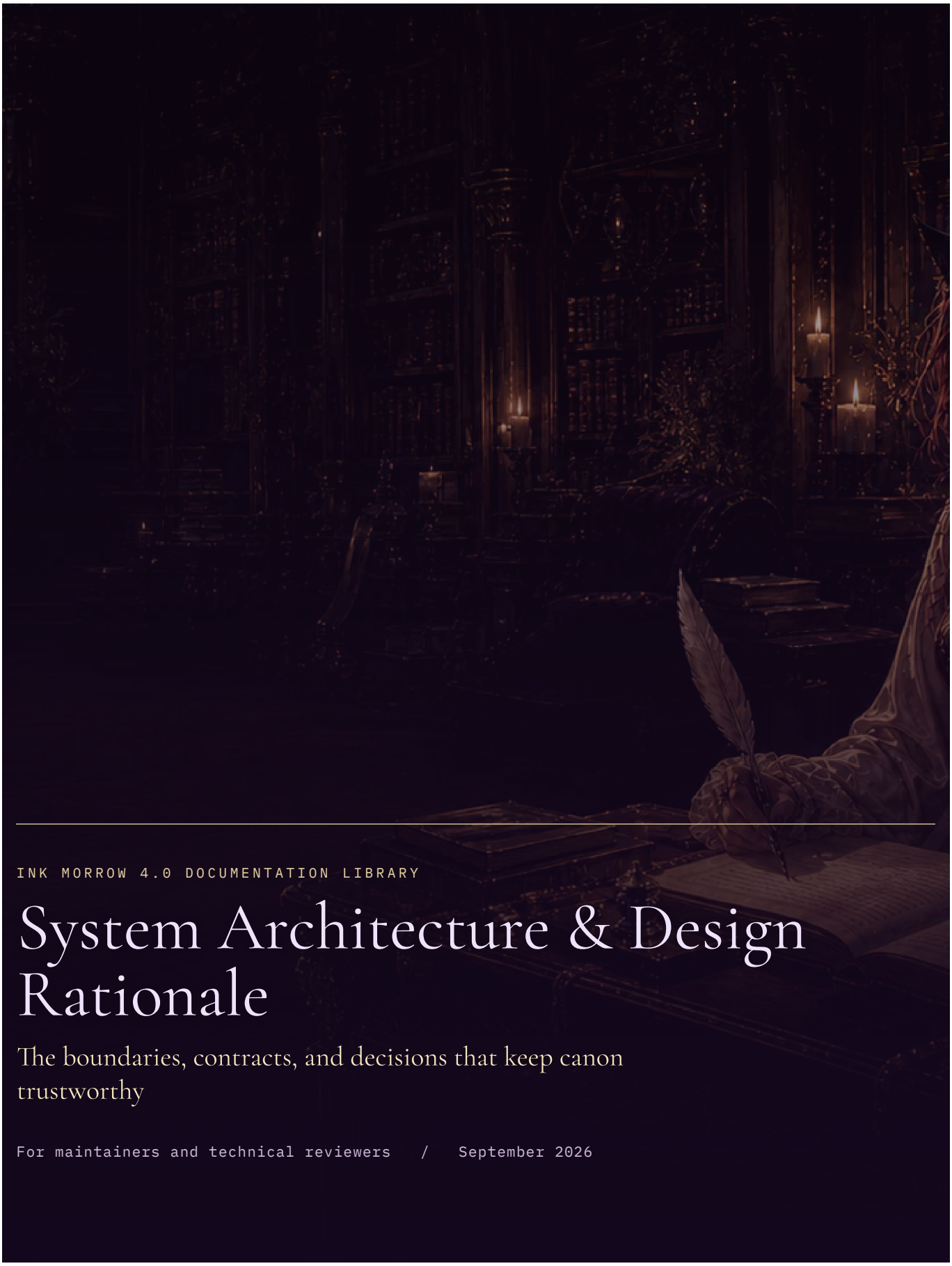

INK MORROW 4.0 DOCUMENTATION LIBRARY

System Architecture & Design Rationale

The boundaries, contracts, and decisions that keep canon trustworthy

For maintainers and technical reviewers / September 2026

System Architecture & Design Rationale

Ink Morrow is a self-hosted writing system whose hardest problem is not text generation. Its hardest problem is keeping authored prose, remembered story truth, revision history, provider spend, and recoverability honest while all of them change at different speeds.

This document explains the current 4.x architecture through boundaries and decisions, including the canonical-storage boundary introduced in 4.1.0. It is intentionally more durable than a source-file tour: modules may move, but the invariants should remain recognizable.

Primary promise: manuscript canon, authoritative story state, and known provider spend advance through explicit, recoverable transactions.

Contents

01 System context

One owner uses a browser to reach one Node.js process. That process serves the frontend and same-origin API, owns one SQLite database, stores media and staging on the same filesystem, and calls only the AI provider explicitly configured by the owner. Public readers see immutable publication snapshots, never the live authoring application.

```
Author / Operator
       |
       | browser, same origin
       v
Ink Morrow Node.js process -----> Configured AI provider
       |                          (only on explicit work)
       +---- SQLite database
       +---- media / audio / transfers
       +---- immutable public snapshots -----> Readers
```

There is no required maintainer cloud, multi-user collaboration service, telemetry collector, or background provider agent. Local manual authoring must remain useful when the provider is unavailable.

02 Container and module map

Backend module	Owns	Must not silently own
`auth`	Owner setup, sessions, password, vault unlock	Provider choice or manuscript content
`providers`	Profiles, capabilities, roles, calls, cost	Canon commits
`catalog`	Reusable world and character templates	Existing manuscript snapshots
`stories`	Manuscripts, hierarchy, pages, revisions, local canon	Provider credentials
`continuity`	Revision-provenanced deltas, fold, corrections, issues	Prose rewriting authority
`writing`	Writer lease, prepared prose, paid operation journal	Publication or image placement
`imagery`	Upload, generation, normalization, placement	Narrative page order
`audio`	Narration and audiobook jobs	Canon state
`publication`	Normalized reading document and adapters	Live story mutation
`sharing`	Immutable snapshots and capability reads	Private API access
`transfer`	Archive plan, stage, verify, commit, restore	Ad-hoc field merge

Frontend features - Library, Desk, Chronicle, Codex, Gallery, Gate, Settings, and the authentication threshold - receive shared services through one composition root. A feature may request another feature's public service; it must not reach through private DOM or module state.

03 Storage identity and migration

The database declares family `ink-morrow-4` and a monotonic schema version. SQLite application/version markers and a migration ledger with checksums prove what has happened to the file.

Startup follows a fail-closed sequence:

1. Resolve the configured storage root and database candidate.
2. Open without applying speculative changes.
3. Prove family, application ID, schema version, and migration checksums.
4. Refuse 3.x, unknown, corrupt, or future identities before mutation.
5. If adopting an earlier 4.0 beta identity, write a full pre-adoption snapshot.
6. Apply known migrations transactionally.
7. Reconcile incomplete jobs and only then listen.

This design rejects filename-based identity. Renaming a file does not make it another schema. It also rejects "best effort" migration because a confident refusal is cheaper than silently corrupting the only manuscript.

Schema 13, shipped with Ink Morrow 4.1.0, makes the hierarchy, immutable page revisions, and revision-bound continuity deltas the only writable sources of manuscript prose and Chronicle memory. Existing page-shaped API responses are assembled through a read-only database view. This removes the earlier duplicate writable mirrors without forcing interface consumers or portable `.inkmorrow` v2 archives to change shape.

04 Domain model

Entity	Durable responsibility
Manuscript	Ordered volumes, local author canon, settings, one active tail
Volume / chapter / page	Hierarchy with stable opaque IDs and scoped order
Page revision	Immutable prose version with parent, kind, direction, timestamps
Prepared page	At most one speculative successor with opaque ID and context fingerprint
Writing operation	Idempotency, expected tail/revision, state, provider result, spend
Continuity delta	One structured memory result per canonical page revision
Continuity correction	Author-owned override with scope and evidence
Continuity issue	Derived warning that later prose may conflict with a correction
Author canon record	Versioned author-created facts, world events, people, and other truths
Template snapshot	Manuscript-local copy of reusable Library source data
Recovery suffix	Expiring package of truncated pages and private dependent state
Asset / placement	Immutable media plus noncanonical anchor between pages
Publication snapshot	Immutable normalized reading copy
Share	Revocable capability record pointing at one snapshot

Stable IDs are independent of display order. Reordering a chapter or inserting art must not change references used by continuity, history, recovery, exports, or links.

05 Canon, display, and author truth

A page has two prose pointers:

- **Canonical revision:** the prose from which continuity was extracted.
- **Display revision:** the prose readers see and exports use.

They are normally equal. A substantive active-tail edit creates a new canonical revision, points both fields to it, invalidates speculative work, and schedules new continuity. A historical copyedit creates only a display revision; it does not pretend that remembered state was recalculated.

Codex overlays three truth layers:

1. **Author Canon** - explicit facts and events created by the author.
2. **Extracted memory** - structured, page-revision-provenanced evidence.
3. **Corrections** - author-owned overrides that preserve the original evidence.

The fold is deterministic. AI may produce a candidate delta or summarize possible impact, but it never receives authority to apply a correction or rewrite prose.

06 The canon transaction

Every canon-changing write executes in one SQLite transaction:

1. Validate owner session, writer lease, and idempotency key.
2. Validate expected manuscript, tail page, canonical revision, and context fingerprint.
3. Write or promote a complete immutable page revision.
4. Update hierarchy and active-tail pointers.
5. Consume or invalidate prepared work exactly once.
6. Create the continuity work item.
7. Commit database changes.
8. Only after commit, schedule extraction and successor preparation.

No partial provider stream is canon. If continuity later fails, valid prose remains canonical while the page displays incomplete memory. This separates literary success from archival success without lying about either.

> ARCHITECTURAL VETO
>
> A green Next Page action may promote only the exact prepared page visible to the author. It cannot fall back to generating different prose. Speculation cannot enter canon or continuity without an explicit commit boundary.

07 Writer concurrency and idempotency

One manuscript has one writer lease with a short heartbeat and visible owning session. Other tabs may read. Conflicting writes are rejected with a reconciliation action; an expired lease is recoverable and is never a permanent lock.

Paid requests carry idempotency identity and expected context. Repeated clicks join the same work or receive the stored result. A provider response that returns after the tail changes becomes superseded and cannot mutate the new context. Known provider usage remains attached to the operation even when canon does not advance.

This makes the operation journal the bridge between unreliable networks and durable authorship.

08 Prepared prose and bounded context

After a successful page, Ink Morrow may prepare exactly one successor. Prepared prose is inert. Its context fingerprint covers the facts that made it valid: tail, revision, direction-sensitive settings, relevant canon, and template snapshots.

Generation context is bounded, not a replay of the complete novel:

- recent display prose;
- compact folded continuity;
- unresolved threads and arcs;
- relevant older evidence;
- manuscript-local templates and Author Canon;
- cast prioritized as Main Character, support, then background; and
- explicit author direction.

In a Main Character-driven story, the Main Character perspective anchor remains present even when the current page does not name them. Bounded context protects cost and provider limits; deterministic prioritization protects narrative identity.

09 Chronicle and Codex architecture

Chronicle is the structural and historical projection: volumes, chapters, pages, revision states, coverage, failures, and recovery records. Codex is the truth and correction projection: foundations, Author Canon, extracted evidence, arcs, threads, corrections, issues, and template differences.

A continuity delta cites its page and canonical revision. The Archivist produces strict versioned JSON. Validation rejects unknown or malformed structure. Failed entries store a specific code, reason, and model so the failure is actionable rather than a generic red badge.

Repair is page-local. It may retry extraction for missing/failed coverage, but it does not replay the whole manuscript or manufacture success. Impact analysis begins with deterministic search across later display revisions and deltas; optional AI can summarize the candidate impact but cannot apply changes.

10 Catalog, local snapshots, and Author Canon

Library worlds and characters are reusable sources. Adding them to a manuscript copies relevant fields into a versioned local snapshot. Later Library edits do not silently alter existing work.

An explicit review computes field-level differences. Accepted fields create a new local snapshot and invalidate incompatible prepared prose. Historical prose and continuity remain unchanged until the author chooses a separate action.

Author Canon is inherently manuscript-local and versioned. The author can create, edit, retire, and restore structured facts, world events, people, places, rules, objects, factions, and other records. Retiring preserves evidence and history; it is not destructive erasure.

11 Media architecture

Art is not a narrative page. An asset records source, content hash, media type, decoded dimensions, normalized derivative, optional title/alt text, and provider provenance. A placement anchors the asset before the first page or after a stable page ID; several placements at one anchor have independent order.

Uploads stream to private staging, enforce byte and decoded-pixel limits, verify signatures and decode, strip metadata, use random storage names, and publish a safe raster derivative. Uploading never calls AI. A reference image crosses the provider boundary only after explicit selection.

Provider-specific prompt sanitation is announce-and-wait: Ink Morrow presents the editable rewrite and cost, then waits for another owner action. It never hides a second generation behind a refusal.

12 Provider and credential boundary

Profiles contain endpoint, declared capability, logical role assignments, and secret reference. APIs never return a submitted secret. Environment credentials are read-only. UI credentials may be session-only or explicitly stored in an encrypted vault.

The vault separates entry encryption from passphrase wrapping. Password changes rewrap the random data key. Plaintext exists only in process memory while unlocked. Terminal password recovery cannot recover the old wrapping key, so it clears stored provider credentials while preserving manuscripts and media.

OpenRouter is the tested default. Capability discovery never silently changes a stored role. An explicit unavailable `CONTINUITY_MODEL` prevents startup instead of falling back to a browser-selected model.

13 Publication and sharing

Every export builds one immutable `PublicationDocument`:

- title, author, and publication metadata;
- ordered volumes, chapters, prose blocks, and scene breaks;
- selected placed art and accessible descriptions; and
- style-independent semantic roles.

DOCX, ODT, RTF, EPUB, PDF, HTML, Markdown, text, and JSON adapters render the same normalized document. This prevents every format from inventing its own manuscript model.

A public share freezes the same document. Snapshot creation is authenticated; snapshot reading is the only unauthenticated route. The high-entropy capability is stored only as a hash. Reads expose no mutable story API, provider action, private identifier, or live update. Replacing manuscript prose never changes an existing share.

14 Portable archives

`.inkmorrow` archives are versioned ZIP containers with a manifest, ordinary JSON, and optional media. They are not raw SQLite files. Planning exposes included entities and exclusions before bytes are streamed.

Import stages outside the catalogue, validates paths, duplicates, symlinks, declaration, counts, expansion ratio, hashes, media, IDs, family, and version, then presents collisions. Commit stages filesystem moves and uses one SQLite transaction. Full replace first writes a persistent safety archive.

Derived indexes, search tables, checkpoints, issues, and caches are rebuilt rather than exported as truth. Credentials, sessions, paid consent, recovery tokens, and share capabilities never travel.

15 Restart and recovery model

The process may stop between any two external observations. Therefore durable jobs record enough state to reconcile without guessing. Startup:

- marks unrecoverable in-flight provider/publication work interrupted;
- removes abandoned staging;
- preserves complete results already committed;
- resumes only work with an explicit safe resumption contract; and
- never invents success from a missing response.

Truncating after page N creates a recovery suffix before removing the canonical tail. Restore is allowed only when the surviving head still matches the recovery fingerprint. Otherwise the suffix can be exported for manual reconciliation. The default recovery window is 30 days.

16 Security design

The primary supported boundary is a single owner on loopback. Authentication still protects against casual local access and deliberate network exposure. State-changing routes require session, CSRF, Origin/Referer, and Host validation. Security headers, strict output encoding, bounded inputs, same-origin design, and encrypted provider secrets provide defense in depth.

Public sharing is a narrower independent surface. HTTPS is required, responses prohibit indexing/framing/sniffing, capability tokens do not enter logs, and the route serves only an allowlisted immutable document.

Provider output, manuscript text, imported archives, and uploaded media are all untrusted input at their respective parser boundaries.

17 Performance and scale

Page turns, history browsing, local editing, and manual organization make no provider call. Queries on the Desk use indexed bounded access rather than whole-story scans. Media and archives stream. Background jobs expose honest pending/running/failed/ready states.

The reference long-manuscript fixture contains 10 volumes, 100 chapters, 3,000 pages, about 1.2 million words, 150 recurring characters, 10,000 memory records, and 500 asset records. Performance decisions are judged against that shape and the reference Android tablet, not only an empty desktop database.

18 Decision register

Decision	Why	Rejected shortcut
Same-origin monolith	Simple self-hosting and security boundary	Distributed services for their own sake
Immutable revisions	History and evidence survive edits	Overwrite page text in place
Separate canonical/display pointers	Honest historical copyediting	Recompute or ignore continuity silently
Explicit local snapshots	Existing stories do not drift	Live-link global templates
Author Canon overlay	Author truth is editable and traceable	Treat extracted AI memory as supreme
Durable paid-operation journal	Idempotency, spend, restart reconciliation	Fire-and-forget provider calls
Art separate from pages	Placement can change without canon	Image pages that shift numbering
One PublicationDocument	Cross-format semantic agreement	Per-adapter story queries
Capability snapshots	Narrow, immutable public surface	Share live authoring routes
Versioned portable archive	Reviewed transfer without raw DB coupling	Copy SQLite into imports

19 Architectural fitness checks

A change should be rejected or redesigned when it:

- stores speculative prose as canon or memory;
- commits an unseen replacement behind Next Page;
- lets global template edits mutate an existing manuscript;
- uses visible order as durable identity;
- hides or duplicates provider spend;
- accepts stale provider output into a changed context;
- stores UI-entered secrets in plaintext;
- serves active uploaded documents from the application origin;
- exposes mutable private APIs through a public capability;
- adds a second independent publication model; or
- requires full-novel AI replay to restore continuity.

The Story is the system's reason for existing. Every architectural convenience remains subordinate to preserving its authorship, evidence, and recoverability.

20 5.0 release-branch foundation preview

The approved 5.0 product is playable fiction for a reader-director outside the cast. The first implementation batch adds `modules/fiction`, independently of the manuscript and optional Play tables. This is a backend foundation, not yet the replacement user interface. The final 5.0 product will use fresh storage; earlier databases and saved stories are not a compatibility requirement.

`fiction_games` owns a title, premise, genre, initial state, active path and optimistic revision. `fiction_branches` owns exact ancestry and a head beat. Each immutable `fiction_beats` row carries readable prose, input attribution, changes and a complete bounded state snapshot. Forking and returning select the snapshot at that moment, including commitments, knowledge and character control. Corrections append history rather than rewriting an earlier beat.

The authenticated `/api/fiction` API creates and reads these stories. Local control, correction, branch and episode operations require the current revision. Paid replies use a separate idempotent request journal and one pending request per story. A validated reply and its state changes commit together. Provider calls receive bounded recent history and relevant facts; there is no automatic per-NPC simulation or whole-story replay. The reader interface, richer scene director and portable 5.0 saves remain later batches, not delivered claims.